Contextual Information-Preserved Architecture Learning for Remote-Sensing Scene Classification

Jie Chen, *Member, IEEE*, Haozhe Huang, Jian Peng, Jiawei Zhu, Li Chen, Chao Tao, and Haifeng Li, *Member, IEEE*

***Abstract*— Convolutional neural networks (CNNs) have recently been widely used in remote-sensing scene classification. Additionally, it is becoming very popular to automatically learn specific CNN architectures for specific data sets. The rich contextual information in high-resolution remote-sensing images (RSIs) is critical to remote-sensing intelligent understanding tasks. However, architecture learning approaches tend to simplify the original data (i.e., resizing images to smaller resolution) for efficiency, yet result in contextual information loss of RSIs. In this article, we proposed a contextual information-preserved architecture learning (CIPAL) framework for remote-sensing scene classification to utilize the contextual information in RSIs as much as possible during the architecture learning process. We introduce channel compression into CIPAL, which can reduce the memory and time consumption of architecture learning and make it possible to construct a larger architecture space. We add potential operators that are rarely used for scene classification tasks (i.e., atrous convolution) into the architecture space to explore unknown architectures that are more suitable for remote-sensing scenes. The experimental results on four remote-sensing scene classification benchmarks indicate that CIPAL learns architectures with less time consumption than similar works, and the newly found architectures outperform popular hand-designed architectures for better use of contextual information in RSIs. Different architectures are good at learning different representations, and our proposed architecture learning method potentially helps us understand which types of representations are crucial for RSI intelligent understanding.**

***Index Terms*— Convolutional neural network (CNN), deep learning, neural architecture search, remote sensing, scene classification.**

I. Introduction

With the advancement of earth observation techniques, the resolution of remote-sensing images (RSIs) (e.g., hyperspectral images [1] and SAR images [2]) has been greatly improved. This improvement allows for more intelligent understanding of RSIs and drives a large number of applications, such as classification of land use and land cover (LULC), RSI retrieval, geographic object detection, traffic planning, urban growth measurement, and hazard monitoring [3]–[7]. These applications mostly demand an intelligent scene classification of RSIs, namely, giving each scene image a specific label (e.g. freeway, river, grassland) [3]. Traditional pixel-level methods can hardly satisfy the need for RSI scene classification. Object-based image analysis (OBIA) [8] and geographic-object-based image analysis (GEOBIA) [9] partition scene images into several segments but carry little semantic information, which is essential for describing the scenes. Due to the complexity of remote-sensing scenes, extracting good semantic information for scene classification is a challenging task.

It is well known that good feature descriptions or representations are critical for obtaining semantic information from RSIs. In other words, feature engineering is one of the most important procedures in remote-sensing scene classification. Handcrafted features (such as scale-invariant feature transform (SIFT) [10] and histogram of oriented gradients (HOG) [11]) were the earliest used for solving remote-sensing scene classification tasks. It was found that designing well-performing handcrafted features is time-consuming and domain knowledge intensive. There are various scenes in RSIs with different spatial resolutions and imaging conditions. Handcrafted features restrict the capacity of feature representations for which they are universal and may not be the best fit for certain RSIs. Recently, deep learning-based [12] methods, especially CNN-based methods, have led to a paradigm shift from manually designing features to automatically learning features from data. CNN-based methods have achieved better performance than traditional methods in most RSI intelligent understanding tasks [13]–[15] are also being used for remote-sensing tasks. Notably, different CNNs may be applicable to different data sets. Nogueira *et al.* [16] found that AlexNet [17] performs better than VGG16 [18] on the UC Merced Land-Use data set [19] but the result is the opposite on the Brazilian Coffee Scenes data set [20]. These CNNs with different architectures are trained under the same conditions, which indicate that the performance variations are mostly dependent on CNN architectures. It is worth noting that the manually designed architectures are also limited by the domain knowledge of network designers and may not fit the data well. Thus, paradigm of automatic learning architectures is becoming popular.

With the development of AutoML technology in natural scenes, learning architectures for specific data sets becomes possible. For the natural scenes, neural architecture

Manuscript received September 13, 2020; revised December 5, 2020 and January 9, 2021; accepted February 27, 2021. Date of publication March 17, 2021; date of current version December 9, 2021. This work was supported in part by the National Natural Science Foundation of China under Grant 41871364, Grant 41571397, Grant 42071427, and Grant 41771458 and in part by using Computing Resources at the High Performance Computing Platform of Central South University. *(Corresponding author: Haifeng Li.)*

The authors are with the School of Geosciences and Info-Physics, Central South University, Changsha 410083, China (e-mail: lihaifeng@csu.edu.cn).

Digital Object Identifier 10.1109/TGRS.2021.3063287

1558-0644 © 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 08:14:10 UTC from IEEE Xplore. Restrictions apply.

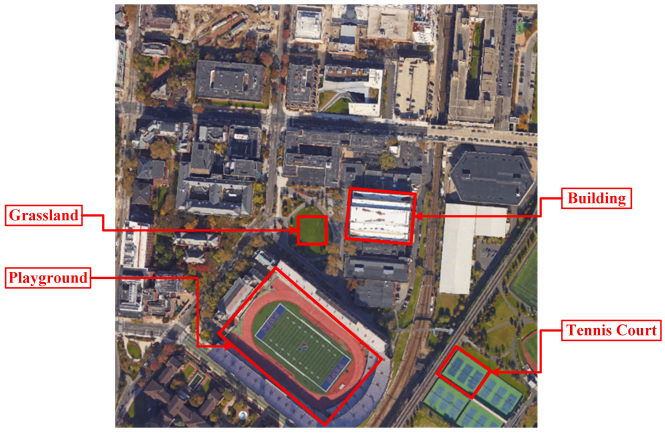

Fig. 1. Example image in the school category from the AID data set. Features of different categories are distributed on the image, such as building, grassland, playground, and tennis court. To correctly classify this image, more contextual information is necessary.

search (NAS) has gained traction as a successful architecture learning paradigm. The earlier NAS approaches are based on reinforcement learning [21], [22] and evolutionary algorithms [23]. And most of them consume enormous computing resources, even reaching thousands of GPU days. Subsequently, one-shot model-based [24]–[26] approaches have emerged to reduce the computational overhead of NAS. One-shot model-based approaches train a supernetwork that contains all operations and share weights to sub-networks, which are ultimately used to validate the performance of NAS. Training a supernetwork and sharing weights means that training time is conserved by not having to train each sub-network. However, since the supernetwork has many times the number of operators than the sub-network, it requires far greater memory consumption than the sub-network. Therefore, one-shot models tend to use proxy tasks such as training on resized or smaller images [26]–[28], which can reduce memory consumption while maintaining accuracy compared with ones without proxy tasks.

However, NAS on RSIs will face new challenges compared to those on natural images because the RSIs contain a richer variety of structurally complex ground objects. As shown in Fig. 1, an image of the school category contains several parts that can be labeled as other categories such as grassland, building, playground, and tennis court. After integrating the semantic information of these parts, the category of this image can be distinguished as school. It means that understanding of remote-sensing scenes relies on good capture of contextual information. For this reason, directly applying NAS approaches that use proxy tasks such as resizing or dividing RSIs to the smaller resolution to remote-sensing scene classification tasks will face contextual information loss and result in a loss of classification performance (As mentioned in Section IV-E Ablation A, learning architectures on resized RSIs will impair the performance of the learned architectures). Therefore, in the remote-sensing scene classification tasks, we need to reduce the loss of contextual information while maintaining architecture learning efficiency. In other words, it is necessary to design a memory and computation efficient architecture learning framework. In addition, a memory and computation efficient architecture learning framework can help us explore a larger architecture space that contains operators that are useful for gaining larger receptive fields and more contextual information. Atrous convolution [29] is widely used in land-cover classification of RSIs [30]–[32] to obtain a larger receptive field, yet it cannot be found in hand-designed CNN backbones that are commonly used in remote-sensing scene classification such as VGG16 [18], ResNet-50 [33], and Googlenet [34]. Inspired by this fact, we add atrous convolution into the architecture space to explore unknown architectures that meet the needs of more contextual information for remote-sensing scene classification.

In this article, we proposed a contextual information-preserved architecture learning framework for remote-sensing scene classification. The major contributions of this article are as follows.

1) We proposed contextual information-preserved architecture learning (CIPAL), an architecture learning framework that can reduce the contextual information loss of architecture learning on RSIs. We perform architecture learning without downsampling the RSIs to take full advantage of the contextual information in the RSIs. The experimental results show that the architectures learned on higher resolution RSIs have better performance.

2) We introduced channel compression mechanism to compress the channels of the feature map that are input to the operators in the CNN architectures through 1×1 convolution. Benefit from this, the memory consumption and time cost of architecture learning have been drastically reduced compared with that of similar works. For example, it takes half an hour to complete the architecture learning on the UCM data set.

3) We expand the architecture space with operators that are rarely used in remote-sensing scene classification tasks to obtain architectures that are better able to capture contextual information. Our experiments show that the newly discovered architectures make better use of the contextual information in RSIs. And the learned architectures outperform most of the popular hand-designed architectures and learned architectures in four remote-sensing scene classification benchmarks. The architectures determine the function of CNNs, and our proposed framework helps in discovering potential well-performing architectures and understanding which representations are important for remote-sensing scene classification tasks.

The rest of this article is organized as follows. Section II introduces the related works. Section III introduces the details of the proposed architecture learning framework. In Section IV, the experimental results with comparative analysis are reported. Finally, the conclusion is provided in Section V.

II. Related Works

In this section, the related works of remote-sensing scene classification and neural architecture search methods are briefly reviewed.

A. Remote-Sensing Scene Classification

Good feature descriptions or representations are critical for remote-sensing scene classification. The features used by the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 08:14:10 UTC from IEEE Xplore. Restrictions apply.

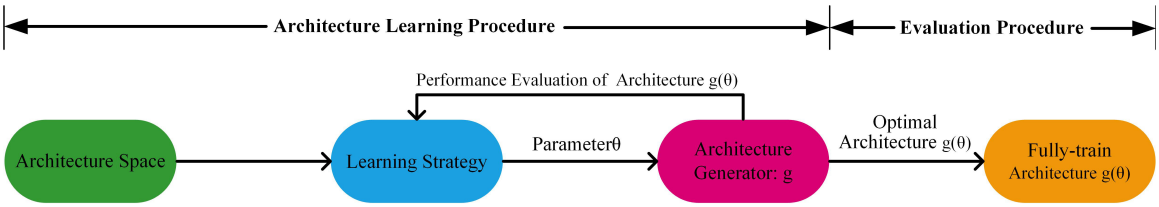

Fig. 2. Illustration of the procedure of learning architecture on a specific data set.

previous works can be roughly divided into designed features and learned features.

Designed features are frequently used in remote-sensing scene classification, such as color histograms and texture descriptors [35], SIFT [10], HOG [4], and bag-of-visual-words [6]. Local and global designed features are combined to obtain better performance on remote-sensing scene classification [36], [37]. With the increasing complexity of remote-sensing tasks, designed features face major challenges. Methods using learned features, especially deep learning features [38]–[44], achieve state-of-the-art performance in most remote-sensing scene classification benchmarks. A great number of methods modify architectures of the deep neural network or introduce new components to CNN architecture for better performance. For example, Zhang *et al.* [45] added CapsNet into a CNN architecture to encode properties and spatial information of features in RSIs. Wang *et al.* [46] added a recurrent attention architecture into a CNN architecture to squeeze high-level semantic features for improved classification performance. Tong *et al.* [47] applied channel-attention component to DenseNet and Cao *et al.* [48] utilized self-attention to improve the capture of critical information in the RSIs. The introduction of more and more advanced deep learning methods, such as GCN [49] and multimodal [50], further improves the ability to understand remote-sensing images.

Adopting new operators, such as atrous convolution, is also a very promising direction. For example, Zhang *et al.* [51] added a multi-dilation pooling module to achieve a multi-scale feature representation. Li *et al.* [52] used atrous convolution as a multiscale feature extraction to generate a new input for CNN and Zhao *et al.* [53] used the similar technique. In these works, atrous convolution is often used as part of a separate module in remote-sensing scene classification tasks. Yet how to utilize it in neural architectures for remote-sensing scene understanding remains an open question.

B. Architecture Learning Mechanism

To design better architectures for a specific task or data set, a paradigm shift from designing architectures to learning architectures for data is imperative. Zoph *et al.* [21] first learned an optimal CNN architecture on the CIFAR-10 data set with reinforcement learning. Further work was devoted to reducing the time consumption of architecture learning and improving the performance of learned architectures such as evolutionary methods [23], [54], random search [55], [56], and gradient-based methods [28], [57]. However, these methods often tend to simplify the original data for efficiency. These approaches may achieve good performance on natural images, but often result in a loss of contextual information on RSIs. Due to the differences between RSIs and natural images, it is problematic to directly introduce architecture learning approaches from the natural image domain into the remote-sensing image domain. Therefore, related works in the field of remote sensing are also emerged. Chen *et al.* [58] proposed an automatic CNN design approach in remote-sensing HSI image classification and Wang *et al.* [59] searched for deep neural networks in remote-sensing recognition tasks. And how to maintain the contextual information in the RSIs as much as possible during the architecture learning procedure still worth studying.

III. Method

CNN-based methods have achieved considerable progress in remote-sensing scene classification by the manual design of CNN architectures through trial and error. However, manually designing CNN architectures for a data set is time-consuming. The architectures need to be modified to fit a new data set. Therefore, learning an optimal deep CNN architecture automatically is worth studying in remote-sensing scene classification.

Learning architectures on a specific data set can be generalized by the procedure shown in Fig. 2. The architecture learning framework contains two procedures: an architecture learning procedure and an evaluation procedure. There are three parts in the architecture learning procedure: architecture space, learning strategy, and architecture generator. The architecture space is a parameter space in which every parameter set θ can be transformed into a CNN architecture $g(\theta)$ by the use of the architecture generator g. A well-defined architecture space will save time during the architecture learning procedure. An efficient learning strategy will then learn the optimal architecture from the architecture space according to the classification performance on the validation data set. When the optimal parameter set θ is obtained, the optimal CNN architecture $g(\theta)$ is also finalized. In the evaluation procedure, $g(\theta)$ will be fully trained to evaluate the performance of the learned architecture. We will present our proposed architecture learning approach, CIPAL, in three parts: architecture space, learning strategy, and architecture generator. The overall CIPAL algorithm is shown in Algorithm 1.

A. Architecture Space With Channel Compression

Architecture space is a collection of all architectures that can be learned. To complete the architecture learning procedure within an acceptable time, we follow the design of architecture space in the one-shot model [24]. In CIPAL, we learned the architecture of the computational cell rather than the entire CNN. A computational cell, illustrated as Fig. 3, is a small

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 08:14:10 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 CIPAL

Architecture Learning Procedure:
Divide the specific data set into training data set D_t and validation data set D_v.
Create the architecture space of the computational cell with channel compression proportion K.
Initialize CNN weights ω and architecture parameter θ.
while not converged **do**
  $\omega \leftarrow \omega - \nabla_{\omega}\mathcal{L}_{D_t}(\omega, \theta)$;
  $\theta \leftarrow \theta - \nabla_{\theta}\mathcal{L}_{D_v}(\omega, \theta)$
end while
Generate architecture $g(\theta)$ with generator g.
Evaluation Procedure:
Divide the specific data set into training data set D_t and testing data set D_{ts}.
Initialize weights ω of architecture $g(\theta)$.
while not converged **do**
  $\omega \leftarrow \omega - \nabla_{\omega}\mathcal{L}_{D_t}(\omega)$
end while
Evaluating the performance on D_{ts}

computing module consisting of n sum nodes, and it can be stacked multiple times to form an entire CNN. Each computational cell has two input tensors, I_1 and I_2, which are the output tensors of previous computational cells or the output tensors of previous convolution layers. Each sum node has several input tensors and one output tensor. Let S_n be the output tensor of the n-th sum node. The set of possible input tensors of the n-th sum node is $\{I_1, I_2, S_1, \ldots, S_{n-1}\}$. The input tensors of the sum node are processed by a mixed operator that contains all of the operators in operator set O. The inputs of operators in the mixed operator are consistent with the input of the mixed operator, and their outputs will be concatenated as the output of the mixed operator. The operator set O, consists of operators (e.g., separable convolution, atrous convolution, average pooling, max pooling and identity) that are all prevalent in modern CNNs.

In mixed operators, outputs of all operators are concatenated as the output of the mixed operator. Output tensors of all sum nodes are concatenated to be the output of the cell. Inspired by [34], we introduce channel compression to the mixed operator to reduce the computational cost during the architecture learning procedure. As shown in Fig. 3, C, the channels of the input feature map for the mixed operator, will be compressed with proportion K by a 1×1 convolution. Thus, the memory consumption of a mixed operator will also be reduced by proportion K. Since low-dimensional feature map can contain a lot of information about a relatively large image patch, the compressed feature map contains the information needed to select the edges of supernet. To keep the dimension of the feature map from decreasing as the network deepens, we use another 1×1 convolutional branch to keep the dimension of the input and output feature maps of the mixed operator consistent. To make better use of the information in the compressed channel, we replace all the convolution operators by residual convolution triplets, illustrated as Fig. 4. The implementation of residual connect is consistent with

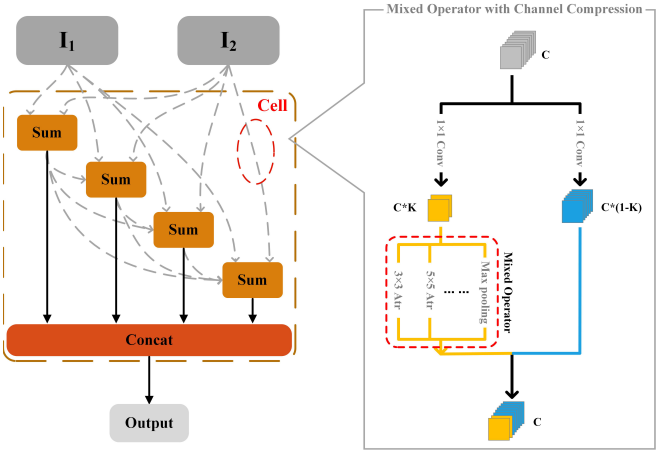

Fig. 3. Computational cell in CIPAL. I_1 and I_2 represent the output of the previous cell and the cell that preceded it. Every gray dotted arrow represents a mixed operator with channel compression described in Section III-A. C, $C*K$ and $C*(1-K)$ represent the number of channels in the feature maps. K is the channel compression proportion. Every sum node of the cell will sum up its input feature maps point by point. Finally, the output feature maps of all sum nodes will be concatenated to serve as the output of this computational cell.

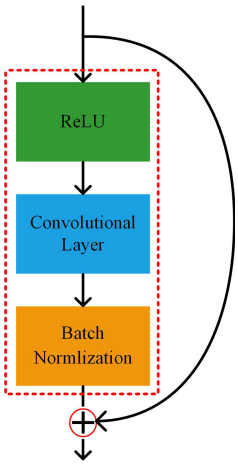

Fig. 4. Illustration of residual convolution triplet. The identities of input are added to the output of the stacked layers.

ResNet [33]. To avoid losing the information contained in other channels, we compress the channels of the input feature map for mixed operators in a proportion of $1-K$ by another 1×1 convolution and concatenate it with the output of mixed operator as the input tensors of sum nodes. Thus, $f^{(i,j)}$, the output of the mixed operators with channel compression of node pair (i, j), can be computed by

$$f^{(i,j)}(x_i, K) = o^{(i,j)}(c_K(x_i)) + c_{1-K}(x_i) \tag{1}$$

where x_i is the input tensor from node i and $o^{(i,j)}$ is mixed operator of node pair (i, j). K is the channel compression proportion, c_K is a 1×1 convolution that can compress the channel of the input tensor with proportion K and c_{1-K} is a 1×1 convolution that can compress the channel of the input tensor with proportion $1-K$.

B. Gradient Descent-Based Learning Strategy

Here, we follow the continuous relaxation described in [27] to parameterize the mixed operator $o^{(i,j)}$, which represents the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 08:14:10 UTC from IEEE Xplore. Restrictions apply.

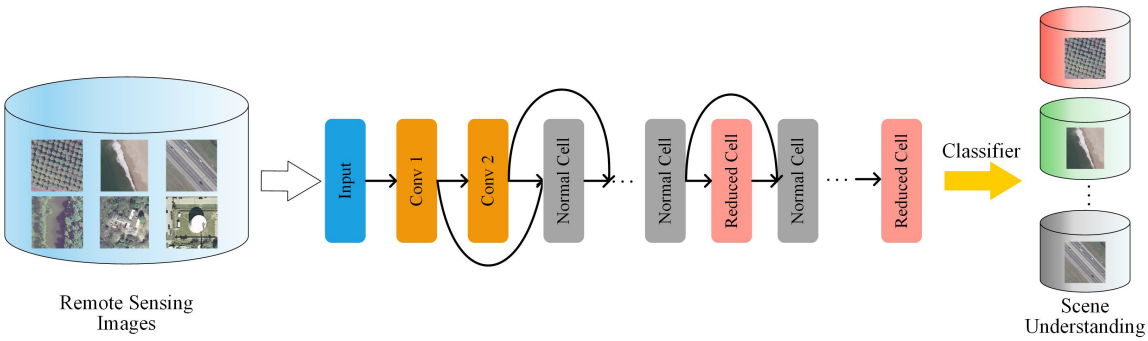

Fig. 5. CNN generated by the architecture generator for remote-sensing scene classification.

mixed operator of node pair (i, j), as a continuous mixed operator $\tilde{o}^{(i,j)}$ over all possible operators in operator set O. Thus, the continuous relaxation of $o^{(i,j)}(c_K(x_i))$ in (1) is

$$\tilde{o}^{(i,j)}(c_K(x_i)) = \sum_{o\in\mathcal{O}} \frac{\exp\left(\theta_o^{(i,j)}\right)}{\sum_{o'\in\mathcal{O}} \exp\left(\theta_{o'}^{(i,j)}\right)} o(c_K(x_i)) \tag{2}$$

where $\tilde{o}^{(i,j)}$ is the continuous mixed operator of node pair (i, j). $\tilde{o}^{(i,j)}(c_K(x_i))$ represents operator $\tilde{o}^{(i,j)}$ being applied to $c_K(x_i)$, the input tensor of mixed operator $o^{(i,j)}$. Additionally, $\theta^{(i,j)}$ is a vector of dimension $|O|$ that determines the operator of node pair (i, j).

In this way, a gradient descent-based method can be introduced into the architecture learning procedure. Let $\mathcal{L}_{\text{train}}$ be the training loss and let $\mathcal{L}_{\text{val}}$ be the validation loss. In the architecture learning procedure, $\mathcal{L}_{\text{val}}$ is used to evaluate the performance of the learned CNN architecture. The learning strategy aims to learn a parameter set $\theta = \{\theta^{(i,j)}\}$ that can minimize the validation loss $\mathcal{L}_{\text{val}}$. Well-trained weights in a CNN are crucial to calculating $\mathcal{L}_{\text{val}}$. Let ω be the weights in the learned CNN architecture. Our learning strategy optimizes ω and θ in the following two steps.

1) Update CNN weights ω by $\nabla_\omega \mathcal{L}_{\text{train}}(w, \theta)$.
2) Update architecture parameter θ by $\nabla_\theta \mathcal{L}_{\text{val}}(w, \theta)$.

After solving the optimization problem

$$\min_{\theta} \mathcal{L}_{\text{val}}(w, \theta) \tag{3}$$

$$\text{s.t. } w = \underset{w}{\text{argmin}}\, \mathcal{L}_{\text{train}}(w, \theta) \tag{4}$$

with gradient descent, optimal parameter set θ is determined at the end of the architecture learning procedure. In addition, two types of cells, normal cell and reduced cell, are learned in the architecture learning procedure. The normal cell keeps the number of channels the same in the input and output tensors. The reduced cell reduces the number of channels by half.

C. Architecture Generator

The architecture generator aims to map every specific parameter set θ into a corresponding CNN architecture at both the cell level and network level. At the cell level, the architecture generator determines the operator between node pair (i, j) according to $o^{(i,j)} = \text{argmax}_{o\in O}\theta_o^{(i,j)}$. Only the two strongest operators will be retained for each node pair (i, j). At the network level, the architecture generator builds the CNN as in Fig. 5. For remote-sensing scene classification, the input images pass two convolution layers (Conv1 is a 3×3 convolution layer with stride 2, and Conv2 is a 3×3 convolution layer with stride 1) in sequence, and the normal cell and reduced cell are stacked several times to complete the CNN.

IV. Experiments

To confirm the effectiveness of our proposed architecture learning framework, we learned the optimal architecture on four remote-sensing scene classification data sets and validated their performance. We train architectures learned by our method and several classic CNN backbones with the same parameter settings in our experiments and compare the performance with our evaluation metrics. In addition, an ablation study is conducted to evaluate the effectiveness of atrous convolution in the architecture.

A. Data Description

In this section, we select four widely used remote-sensing scene classification data sets (UC Merced Land-Use [19], AID [39], NWPU-RESISC45 [3] and PatternNet [60]) to test the effectiveness of our proposed method.

1) UC Merced Land-Use Data set: The UC Merced Land-Use data set is one of the most widely used remote-sensing scene classification data sets and includes 2100 aerial RSIs in the RGB color space with size of 256×256 pixels. The data set is divided into 21 scene types, each of which contains 100 images.

2) Aerial Image Data set: AID is more complex than the UC Merced Land-Use data set as a result of containing more scene types and greater image resolution. In the AID, 10,000 images in the RGB color space with a size of 600×600 pixels are divided into 30 classes.

3) NWPU-RESISC45 Data set: NWPU-RESISC45 is a larger data set than UC Merced Land-Use and AID, containing 31 500 images that are divided into 45 scene types. Each type consists of 700 RSIs in the RGB color space with a size of 256×256 pixels.

4) PatternNet Data set: PatternNet is collected from high-resolution RSIs with a size of 256×256 pixels. It contains 38 classes with 800 images per class. Different from the above three data sets, PatternNet covers fewer background regions than UCM, AID, and NWPU45, and the categories in PatternNet are not strongly dependent on contextual information. For

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 08:14:10 UTC from IEEE Xplore. Restrictions apply.

this reason, we believe that scene classification in PatternNet relies less on contextual information than that in the other three data sets.

B. Evaluation Metrics

We evaluate our proposed CIPAL from two aspects: effectiveness and efficiency. For the data sets with similar numbers of images in each class, the average accuracy (AA) will be close to the overall accuracy (OA). Thus, we excluded AA from the evaluation metrics in our results. The OA and confusion matrix(CM) were used to determine the effectiveness of our method. To evaluate the efficiency of CIPAL, we introduce memory consumption and time cost as evaluation metrics.

1) Overall Accuracy: OA is one of the most widely used evaluation metrics. It is defined as the number of all correctly classified samples divided by the total number of samples.

2) Confusion Matrix: In the confusion matrix, every element is generated by counting test samples according to their actual categories and predicted categories. The rows represent actual categories, and the columns represent the predicted categories.

3) Memory Consumption: We recorded the GPU memory consumption as the memory consumption in our experiments.

4) Time Cost: We separately recorded the time spent on architecture learning and architecture evaluation to evaluate the time cost in our experiments.

C. Implementation Details

In this section, we explain the parameter settings for our experiments. Our experiments can be divided into two primary parts: an architecture learning procedure and an evaluation procedure. First, we learn the optimal architecture through our architecture learning procedure, and then, we evaluate the learned optimal task-specific architecture on the corresponding data sets and compare the performance with that of both manually designed and automatically learning CNN models under the same training settings, including AlexNet [17], VGG16 [18], ResNet-50 [33], Googlenet [34], InceptionV3 [61], DenseNet169 [62], DARTS [27], and P-DARTS [28], to explain the effectiveness of our method. We also compare it with similar work in the remote-sensing community. All experiments are based on PyTorch on a single NVIDIA Tesla V100. The memory consumption and time cost are recorded to evaluate the efficiency of CIPAL. In the architecture learning procedure, we divide a data set into two isometric parts: a training set and a validation set. The training set is used to learn the CNN architecture, and the validation set is used to evaluate the performance of the learned architectures. The architecture with the best performance on the validation set is chosen as the optimal architecture. In the evaluation procedure, the division of data sets depends on the settings of the experiments.

1) Architecture Learning Procedure: The operator set O used in our architecture space contains residual 3×3 depth-wise separable convolution triplet, residual 5×5 depth-wise separable convolution triplet, residual 3×3 atrous convolution triplet, residual 5×5 atrous convolution triplet, 3×3 average pooling, 3×3 max pooling and identity. Among them, separable convolution and pooling are the basic components of CNNs. The use of separable convolutions instead of convolutions makes the architecture learning procedure more efficient. Atrous convolution [29] (the same as dilated convolution [63]) is widely used in land-cover classification of RSIs to obtain a larger receptive field, so we add it into the architecture space to explore unknown architectures that meet the needs of more contextual information for remote-sensing scene classification. And identity operator makes it possible for a node's feature map to be passed to the next node without change. For an acceptable computing time of the architecture learning procedure, we set the number of sum nodes in a computational cell to 4, and every sum node only aggregates 2 outputs from previous nodes. The outputs of these four sum nodes are concatenated together as the output of the computational cell. Following [22], we search two types of computational cells for better performance in our experiments. In a normal cell, the output resolution of the feature map remains the same as the input resolution, and the stride of all operators is set to 1. In a reduced cell, the stride of all operators is set to 2. The number of reduced cells used in our architecture is set to 2 in our experiments.

As described in section III, when the learning strategy finds a set of parameters, the architecture generator maps it to computational cells and then stacks these cells several times to complete the entire CNN architecture. We devoted multiple attempts to obtaining the best number of cells in the optimal CNN architecture for every data set; i.e., the number of cells was set to 4 for the UC Merced Land-Use data set, 6 for AID, and 10 for the NWPU-RESISC45 and PatternNet date sets. Then, we trained the CNN and architecture parameters in alternating fashion from scratch. Because of the introduction of channel compression, CIPAL greatly reduces the time cost and memory consumption. Thus, we can learn the architecture with larger resolution images in a larger batch size. In the architecture learning procedure, images are resized to 224×224, which is consistent with the evaluation procedure. Due to the difference in the amount of data in each data set, we use a small batch size for data sets with a small number of images and a large batch size for data sets with a large number of images, thus ensuring sufficient iterations for model convergence. The batch size is set to 64 for the UCM data set and AID, 128 for the NWPU45 and PatternNet data sets. The memory consumption can be afforded by one GPU. The other training parameters are as follows. We trained our CNN models using stochastic gradient descent with an initial learning rate of 0.025, momentum of 0.9, and weight decay of 3×10^{-4} for 50 epochs. We trained our architecture parameters with a learning rate of 3×10^{-4} and weight decay of 1×10^{-3}. For the other architecture learning methods used for comparison, we used the same epoch as in our method and the remaining hyperparameters were consistent with their original articles.

2) Evaluation Procedure: In the evaluation procedure, we trained the architectures learned by CIPAL and classic CNN architectures with the same settings. The optimal cells were stacked the same number of times as in the architecture

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 08:14:10 UTC from IEEE Xplore. Restrictions apply.

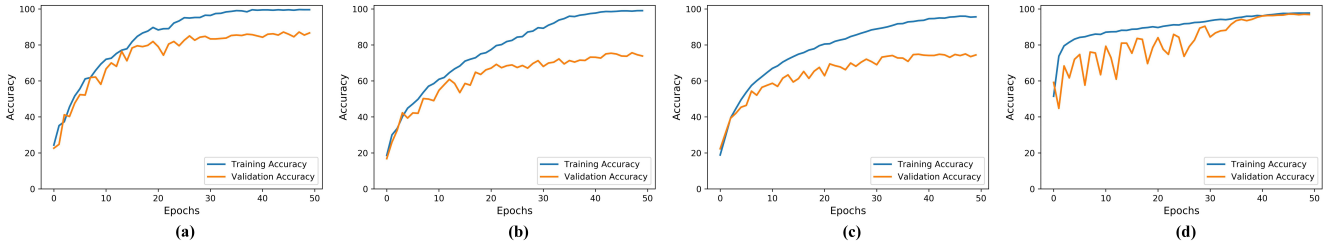

Fig. 6. Training and validation accuracy during the architecture learning procedure. (a) Training and validation accuracy in the UCM experiment. (b) Training and validation accuracy in the AID experiment. (c) Training and validation accuracy in the NWPU45 experiment. (d) Training and validation accuracy in the PatternNet experiment.

learning procedure. The images were all resized to 224 × 224 with random horizontal flip and color jittering before inputting to the CNNs. Our architectures were fully trained [16] with stochastic gradient descent, an initial learning rate of 0.1 with a decay of 0.97 in every epoch, and momentum of 0.9. Considering that the sizes of different data sets are very different, we set a smaller batch size for small data sets and a larger batch size for large data sets. The small batch size enables more iterations in the training process of the CNN. Specifically, we chose a batch size of 16 for the UCM data set, 64 for AID, and 128 for the NWPU45 and PatternNet data sets. Every model was trained for 300 epochs to ensure convergence. For all other methods used for comparison, we used the same hyperparameters as our method.

D. Experimental Results

In this section, we demonstrate the architectures learned by CIPAL and compare the performance of these architectures with that of classic CNN architectures widely used in remote-sensing scene classification.

1) Efficiency Analysis of CIPAL: For a comprehensive analysis of the efficiency of CIPAL, we analyze both the efficiency of the architecture learning procedure and the efficiency of architectures learned by CIPAL. We applied our architecture learning procedure on the four remote-sensing scene classification benchmarks. As shown in Fig. 6, we learned the optimal architectures for 50 epochs. During the architecture learning procedure, the training accuracy and valid accuracy continue to increase, finally approaching convergence at the 50th epoch. The convergence of the validation accuracy on a specific data set indicates that CIPAL has learned the optimal architecture on the data set. The validation accuracy converges faster for the two simpler data sets: UCM and PatternNet.

We performed the efficiency analysis on the UCM data set as an example. We learned the optimal architectures on the UCM data set with different experiment sets and recorded their search cost and memory consumption in the architecture learning procedure. The results are shown in Fig. 7. We used CIPAL without channel compression as a baseline for architecture learning, and 1.62 GPU hours were required to finish the architecture learning procedure, with 12.22-GB memory consumption on 32 × 32 resolution images. The batch size was set to 64, enabling our experiments to be performed on a single GPU. Experiments have shown that after the introduction of channel compression, the memory

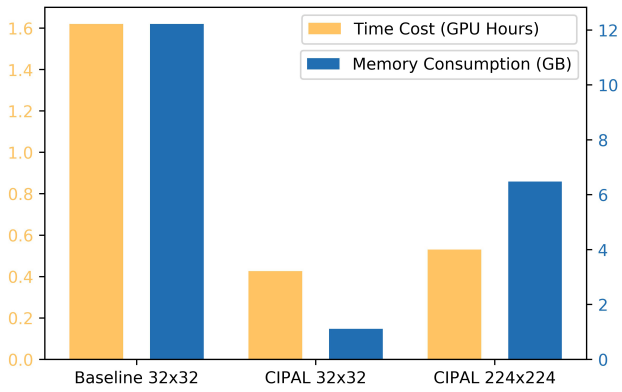

Fig. 7. Time cost and memory consumption comparison between different experiment sets in the architecture learning procedure on the UCM data set. Baseline 32 × 32: CIPAL model without channel compression, learning architecture on the images with 32 × 32 resolution. CIPAL 32 × 32: CIPAL model, learning architecture on the images with 32 × 32 resolution. CIPAL 224 × 224: CIPAL model, learning architecture on the images with 224 × 224 resolution. The batch sizes of the above three experiments are all 64.

consumption of the architecture learning procedure dropped to 1.11 GB when learning architecture on 32 × 32 resolution images and 6.48 GB when learning architecture on 224 × 224 resolution images. And the time cost of architecture learning dropped to 0.43 GPU hours on 32 × 32 resolution images and 0.53 GPU hours on 224 × 224 resolution images. Increasing the resolution of input images to 224 × 224 will increase the time cost and memory consumption, but it is still more efficient than the baseline. We can conclude that the lower memory consumption of CIPAL will make it possible to take advantage of a larger batch size with larger resolution images. This advantage makes better use of the contextual information in the RSIs while reducing the time cost of the architecture learning procedure.

Efficiency comparisons of architectures on the UCM data set are shown in Table I. The parameters of the architectures and the time costs of training architectures from scratch are compared. For the manually designed CNN architectures, the time cost represents the time required to train the CNN from scratch with a single GPU. For the automatically designed CNN architectures, the time required to learn the optimal architectures (the number before the plus sign) is also recorded. The computational complexity of architecture learning approaches can be measured by the time cost of the architecture learning procedure. It is easy to observe that the number of parameters of architectures learned by CIPAL, 1.53M, is similar to

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 08:14:10 UTC from IEEE Xplore. Restrictions apply.

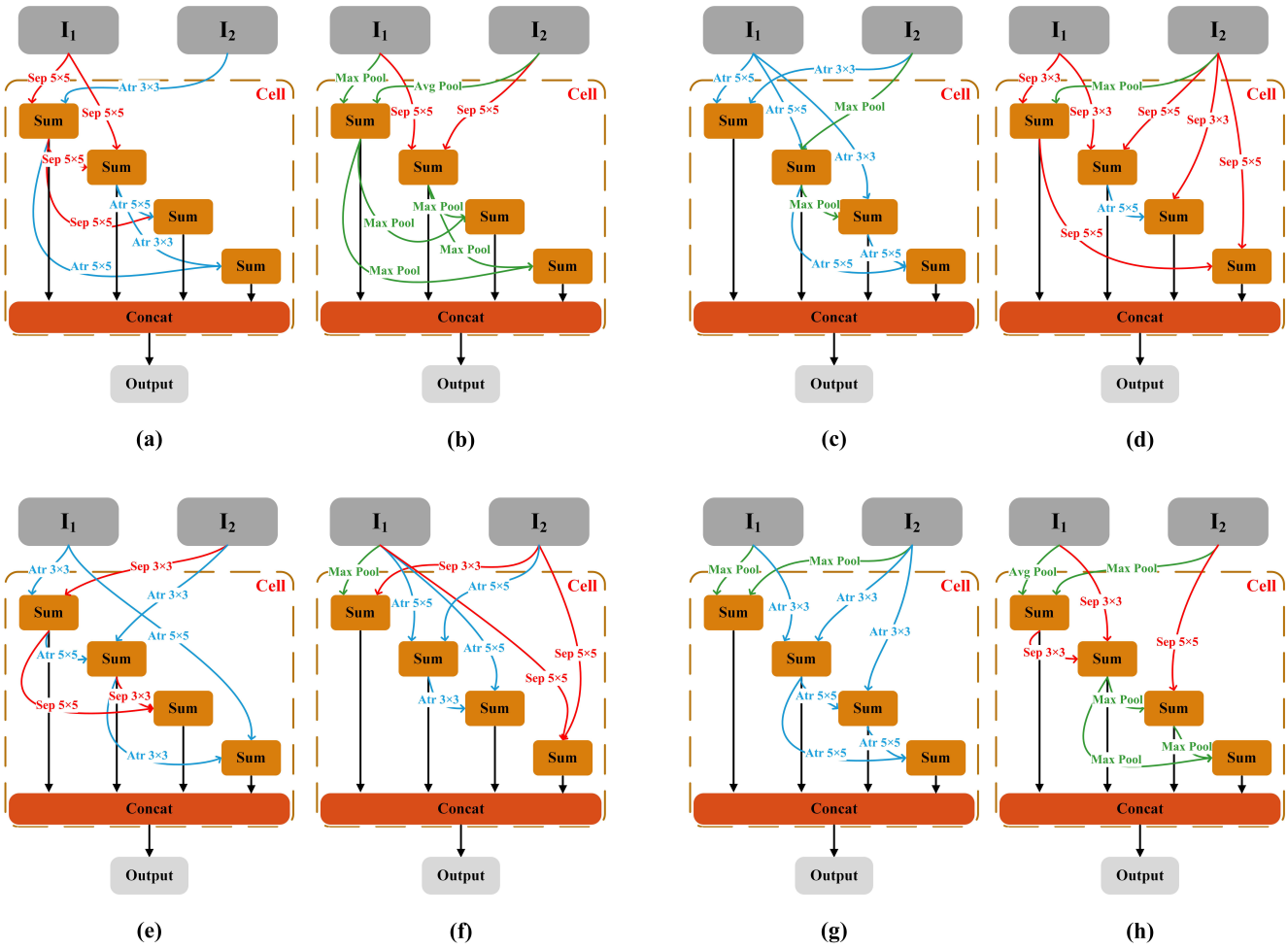

Fig. 8. Optimal computational cell learned by CIPAL. (a) Optimal normal cell on the UCM data set. (b) Optimal reduced cell on the UCM data set. (c) Optimal normal cell on the AID. (d) Optimal reduced cell on the AID. (e) Optimal normal cell on the NWPU-RESISC45 data set. (f) Optimal reduced cell on the NWPU-RESISC45 data set. (g) Optimal normal cell on the PatternNet data set. (h) Optimal reduced cell on the PatternNet data set.

TABLE I
EFFICIENCY COMPARISON OF ARCHITECTURES ON THE UCM DATA SET

Architecture	Params(M)	Time Cost (GPU-hours)	Design Method
AlexNet	61.10	- + 1.65	manual
VGG16	138.36	- + 2.50	manual
Googlenet	6.80	- + 1.73	manual
InceptionV3	27.16	- + 3.12	manual
ResNet-50	25.56	- + 2.01	manual
DenseNet	14.15	- + 2.30	manual
DARTS	1.11	1.18 + 1.75	automatic
P-DARTS	1.76	1.16 + 1.81	automatic
RSNet [59]	1.22	24.0 + 1.73	automatic
CIPAL	1.58	0.53 + 1.77	automatic

architectures learned by other architecture learning methods and much smaller than the classic architectures. Thanks to the smaller number of parameters, architecture learned by CIPAL can be trained from scratch faster than manually designed CNNs other than AlexNet. The introduction of the channel compression mechanism greatly increases the efficiency of the architecture learning compared to similar work in the field of remote sensing. CIPAL required 0.53 GPU hours to learn the optimal architecture on the UC Merced data set; in comparison, RSNet [59] required approximately one day to learn an architecture. Compared to other architecture learning methods, i.e., 1.18 h for DARTS and 1.16 h for P-DARTS, the architecture learning efficiency of CIPAL is still superior.

2) Effectiveness Analysis of CIPAL: To demonstrate the effectiveness of CIPAL, the optimal architectures learned on the four remote-sensing benchmarks are trained on the corresponding data sets, and their accuracies are compared to those of manually designed architectures under the same training conditions. Since the learned architectures could not be used with a pretrained model, we trained all architectures from scratch to ensure a fair comparison. It is worth noting that compared with the fine-tuned strategy, the fully trained strategy was pruned for overfitting and resulted in a performance decrease [16].

The optimal cells learned by CIPAL in the architecture learning procedure are illustrated in Fig. 8. We learned two different types of computational cell, a normal cell and a reduced cell, for every data set. The sum in the images represents the sum node. Sep 3×3 and 5×5 represent 3×3 and 5×5 depth-wise separable convolution residual triplets, respectively. Atr 3×3 and 5×5 represent 3×3 and 5×5 atrous convolution residual triplets, respectively. Max pool represents 3×3 max pooling, and the black arrows without any text represent identity. We find that atrous convolution is

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 08:14:10 UTC from IEEE Xplore. Restrictions apply.

TABLE II
PERFORMANCE COMPARISON BETWEEN METHODS ON FOUR REMOTE-SENSING SCENE CLASSIFICATION BENCHMARKS

Methods	UCM		AID		NWPU45		PatternNet	
	TP%		TP%		TP%		TP%	
	50%	80%	50%	80%	50%	80%	20%	50%
AlexNet	86.38 ± 0.86	89.52 ± 0.78	83.84 ± 0.78	87.65 ± 0.41	84.53 ± 0.40	87.54 ± 0.63	95.12 ± 0.10	97.69 ± 0.02
VGG16	88.27 ± 0.74	94.88 ± 0.97	88.20 ± 0.43	90.35 ± 0.46	88.38 ± 0.34	92.17 ± 0.14	95.07 ± 0.20	97.79 ± 0.09
Googlenet	87.21 ± 1.06	93.36 ± 0.83	89.62 ± 0.46	90.35 ± 0.52	89.31 ± 0.15	92.45 ± 0.28	95.21 ± 0.19	98.03 ± 0.16
InceptionV3	89.71 ± 0.89	95.05 ± 1.20	89.64 ± 0.30	92.70 ± 0.35	89.68 ± 0.27	92.88 ± 0.15	95.74 ± 0.31	98.40 ± 0.11
ResNet-50	89.24 ± 0.77	95.49 ± 0.65	88.82 ± 0.78	91.20 ± 0.52	88.26 ± 0.26	92.34 ± 0.26	95.96 ± 0.11	98.89 ± 0.06
DenseNet	89.68 ± 0.73	94.52 ± 0.57	88.86 ± 0.87	90.95 ± 0.31	88.69 ± 0.52	92.71 ± 0.17	95.89 ± 0.12	98.31 ± 0.06
DARTS	88.64 ± 0.91	93.51 ± 0.50	90.54 ± 0.50	92.18 ± 0.28	91.10 ± 0.30	93.07 ± 0.30	97.45 ± 0.20	99.18 ± 0.05
P-DARTS	89.46 ± 1.11	93.33 ± 0.48	90.67 ± 0.22	92.38 ± 0.12	91.98 ± 0.31	93.21 ± 0.17	97.87 ± 0.11	99.11 ± 0.06
RSNet [59]	-	**96.78 ± 0.60**	-	-	-	92.66 ± 0.42	-	-
CIPAL	**91.96 ± 0.91**	96.58 ± 0.76	**91.22 ± 0.83**	**93.45 ± 0.31**	**92.32 ± 0.31**	**94.64 ± 0.25**	**97.96 ± 0.19**	**99.26 ± 0.10**

most widely used in the optimal normal cells for the four data sets. Atrous convolution is known for its larger field of view of filters for capturing larger contextual information. Architectures with atrous convolution may achieve better performance by meeting the demand for remote-sensing scene classification with larger contextual information. Most pooling operators appear in the reduced cells, corresponding to the downsampling function of the reduced cell.

In the evaluation procedure, we trained the optimal architectures on the corresponding data sets. We also compared the accuracy with RSNet [59], the state-of-the-art architecture learning approach in remote-sensing scene classification, and manually designed architectures. The result is shown in Table II. For all experiments, we repeated the experiment five times and calculated the mean and variance to reduce the effect of randomness on the results. Considering that more data are required to train the architecture from scratch, we separately conducted the experiments at 50% and 80% of the training set proportion. It is important to note that since the PatternNet data set is less complex than other data sets, when using a training ratio of 80%, the validation accuracy of each architecture is close to 100%, thus making comparison difficult. Therefore, for PatternNet, we have chosen two training ratios of 20% and 50%. For the DARTS and P-DARTS, we trained the architectures they learned on four benchmarks with the parameters described in Sec. IV-C. For RSNet, since the original article did not perform experiments on AID and PatternNet and did not perform experiments on the 50% training set proportion of UCM and NWPU45, we are unable to make comparisons for these sections.

AlexNet does not perform as well as other architectures on all data due to its older design. Other manually designed architectures perform similarly on all four benchmarks. For the UC Merced Land-Use data set, in the manually designed architectures, InceptionV3 and ResNet-50 achieved the best performance at 50% and 80% of the training ratio, respectively. The architecture learned by CIPAL outperformed all manually designed architectures with the two training ratios. For the training ratio of 50%, the accuracy of the learned architecture exceeds that of the manually designed architectures by approximately 2-4%. For the training ratio of 80%, the accuracy improvement declined to approximately 2%. This result may be because the superiority of the architecture is more likely to manifest when there are fewer training data. At 80% of the training proportion, we achieved results similar to RSNet (0.2% poorer), but our approach learns the architecture more efficiently than RSNet (0.5 h versus 24 h, respectively). For the AID, CIPAL achieved similar results; it outperformed the manually designed structure by approximately 3% in terms of accuracy at 50% of the training ratio and 2% at 80% of the training ratio. For the NWPU45 data set, due to the increased quantity of data, CIPAL learns that the optimal architecture on a specific data set and will have a better ability to make use of contextual semantic information in RSIs. The optimal architecture on NWPU45 outperformed the manually designed architectures by approximately 3% in terms of accuracy at 50% of the training ratio and by 2% at 80% of the training ratio, and it achieved a 2.02% improvement in accuracy compared to that of RSNet. For the PatternNet data set, due to the simplicity of the data set, the optimal architecture on PatternNet outperformed the manually designed architectures by approximately 2% in terms of accuracy at 50% of the training ratio and by 1% at 80% of the training ratio. The two architecture learning methods in natural scene, DARTS and P-DARTS, achieve slightly lower performance than our method on the four benchmarks but higher than that of manually designed architectures. This also demonstrates the superiority of the automatically learning paradigm over manually design.

We can conclude from the experimental results that CIPAL can obtain optimal architectures on specific data sets with an efficient architecture learning procedure. By reducing the loss of contextual information in the architecture learning procedure, the learned architectures surpassed most manually designed architectures in classification accuracy and CIPAL is more efficient than similar architecture learning methods.

E. Ablation Study

Reducing the contextual information loss during the architecture learning procedure is the main motivation for our approach. In this section, we illustrate experimentally that architecture learning on low-resolution or resized images introduced by previous approaches to mitigate the massive memory

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 08:14:10 UTC from IEEE Xplore. Restrictions apply.

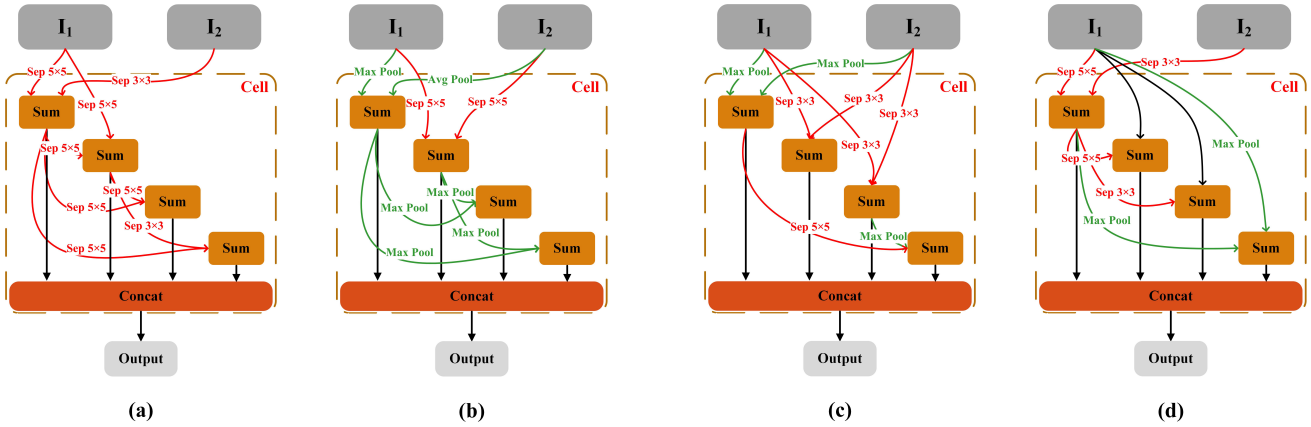

Fig. 9. Cells on the UCM data set in ablation study. (a) Normal cell on the UCM data set with ablation B. (b) Reduced cell on the UCM data set with ablation B. (c) Normal cell on the UCM data set with ablation C. (d) Reduced cell on the UCM data set with ablation C.

TABLE III

PERFORMANCE COMPARISON BETWEEN MODELS OF CIPAL AND MODELS WITH ABLATION

Methods	UCM		AID		NWPU45		PatternNet	
	TP%		TP%		TP%		TP%	
	50%	80%	50%	80%	50%	80%	20%	50%
Ablation A	88.63 ± 1.13	93.43 ± 0.71	87.00 ± 0.75	91.12 ± 0.50	89.73 ± 0.39	93.67 ± 0.24	97.72 ± 0.17	99.12 ± 0.12
Ablation B	90.52 ± 1.28	95.64 ± 0.51	89.71 ± 0.85	93.35 ± 0.78	91.93 ± 0.27	94.59 ± 0.34	97.93 ± 0.14	99.23 ± 0.08
Ablation C	89.17 ± 0.89	94.99 ± 0.83	89.24 ± 1.02	93.08 ± 0.55	91.95 ± 0.26	94.17 ± 0.34	97.81 ± 0.16	99.15 ± 0.06
CIPAL	**91.96 ± 0.91**	**96.58 ± 0.76**	**91.22 ± 0.83**	**93.45 ± 0.31**	**92.32 ± 0.31**	**94.64 ± 0.25**	**97.96 ± 0.19**	**99.26 ± 0.10**

consumption results in a loss of contextual information and thus degraded performance of the discovered architectures. For the observation that all optimal architectures found by our architecture learning procedure contain an atrous convolution operator, the larger contextual information identified by atrous convolution may be crucial for remote-sensing scene classification. To further verify our conclusion, we designed two other ablation experiments. Overall, we performed ablation experiments in three ways.

1) Ablation A: To illustrate the contextual information loss that would result from performing architecture learning on resized images, we resized the images to a resolution of 32 × 32 and then performed CIPAL on resized images. The learned architectures were trained from scratch according to the settings in Sec. IV-C. The performance comparison between the architectures learned on images of 32 × 32 resolution and 224 × 224 resolution is shown in Table III. For all experiments, we repeated the experiment five times and calculated the mean and variance to reduce the randomness of the results. The experimental results show that the architectures learned on resized images exhibited a performance decrease from 3.33% to 0.14%, and the losses of accuracy on UCM, AID, and NWPU45 are greater than on PatternNet. This result indicates that since scene classification on PatternNet is less dependent on contextual information than on the other three data sets, accuracy on PatternNet is less affected by learning architecture on resized images.

2) Ablation B: In ablation B, to verify the role of atrous convolution in the learned architectures, we replaced all of the atrous convolutions in learned optimal cells by separable convolution with the same size and trained these architectures with the same training settings described in Section IV-C. As shown in Fig. 9 (a) and (b), we replaced all atrous convolution with separable convolution in the optimal cell of UCM, as illustrated in Fig. 8 (a) and (b). The performance comparison between the architectures with and without atrous convolution is shown in Table III. Obviously, all of the architectures without atrous convolution exhibited a performance decrease from 1.44% to 0.03%, which indicates that the architectures with atrous convolution can learn a better representation than without it.

3) Ablation C: Another ablation experiment we conducted excluded atrous convolutions in the architecture space, and then repeated the previous architecture learning procedure to learn the optimal architecture with the new architecture space. The newly learned architecture on the UCM data set is shown in Fig. 9 (c) and (d) as an example. We then trained these architectures with the same training settings described in Section IV-C. The experimental results are shown in Table III. The architectures learned by CIPAL after excluding atrous convolution from architecture space exhibited a performance decrease from 2.79% to 0.11% compared to what was not excluded. This performance decrease is greater than that in ablation B. On the one hand, this result shows that architectures with atrous convolution offer better performance, and on the other hand, it indicates that the architecture space with atrous convolution will lead to a better architecture learning procedure. The function of the architecture is related not only to the operator but also to the way in which the operators are connected to each other. The operators in ablation B are connected in a way that is superior to that in ablation C, and therefore, the architectures of ablation B exhibit better performance when using the same operators as ablation C.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 08:14:10 UTC from IEEE Xplore. Restrictions apply.

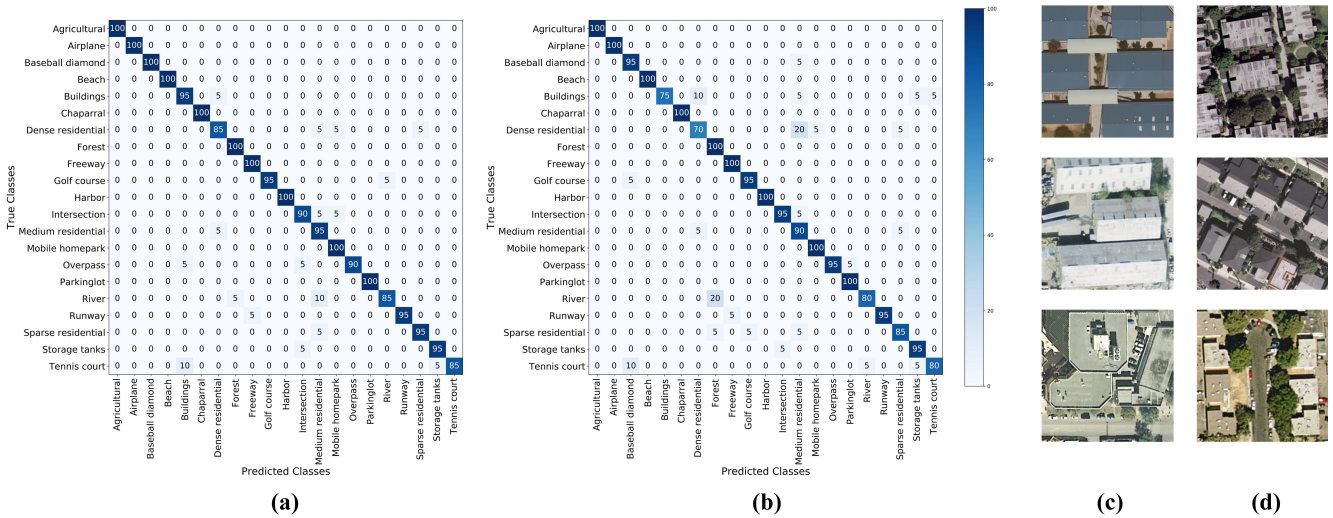

Fig. 10. Confusion matrix and image examples of the UCM data set. (a) Confusion matrix of CIPAL. (b) Confusion matrix of ablation C. (c) Examples in building class. (d) Examples in dense residential class.

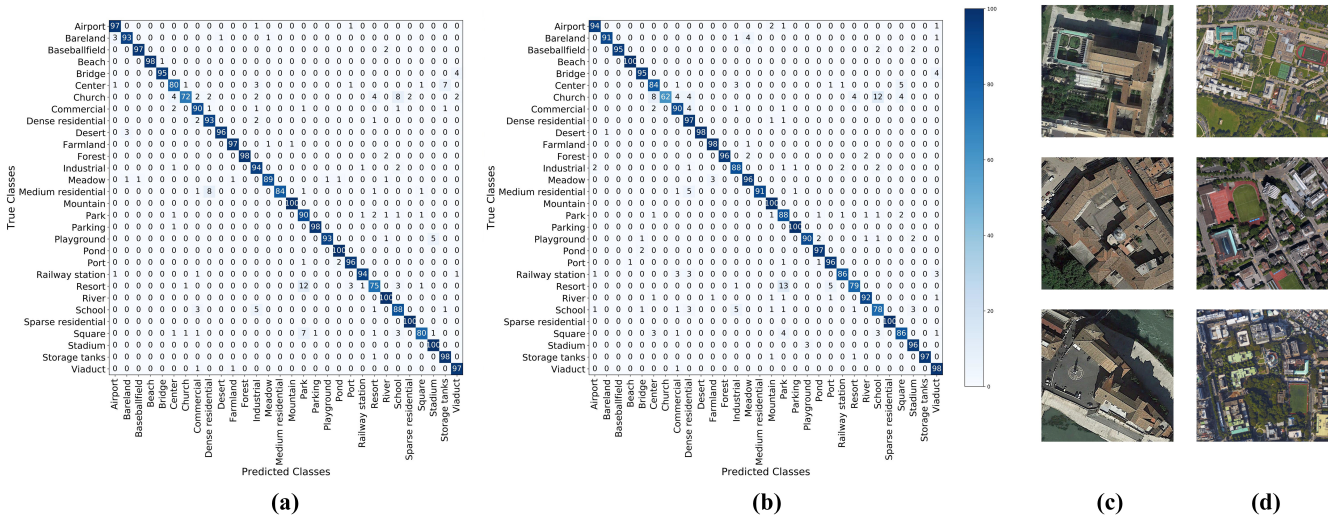

Fig. 11. Confusion matrix and image examples of the AID. (a) Confusion matrix of CIPAL. (b) Confusion matrix of ablation C. (c) Examples in church class. (d) Examples in school class.

The confusion matrix is used to further illustrate the difference between architectures with atrous convolution compared with architectures without atrous convolution. We compared the confusion matrix of CIPAL in Table II with the confusion matrix of experiments in ablation C. For all experiments, repeated five times, we plot the confusion matrix based on the results of one of these experiments. As shown in Fig. 10 (a) and (b), for the UCM data set, the degradation of accuracy primarily occurs in the building and dense residential categories for architectures without atrous convolution compared to those with atrous convolution. For the confusion matrix shown in Fig. 10 (b), we can observe that more images that fall into the dense residential category are misclassified as medium residential, and more images of the building category are misclassified as dense residential. Examples in the respective building and dense residential classes are shown in Fig. 10(c) and (d). We can find that the dense residential category contains more buildings in an image than the building category. Without higher level contextual information obtained by atrous convolution, precisely distinguishing between the two categories will be difficult.

Similar phenomena are also observed in the AID and NWPU45 data sets. As shown in Fig. 11, for the AID, the decrease in accuracy occurred in the church and school categories. Information about other features around the buildings is also important to properly classify these categories. Thus, the larger receptive field provided by atrous convolution will play a role. For the NWPU45 data set, as shown in Fig. 12, the situation is similar to that of the UCM data set. Images in both the commercial area and dense residential categories contain large numbers of buildings, and higher level contextual information will help to correctly distinguish between these categories. For PatternNet, the results are shown in Fig. 13, and the classification accuracy is not significantly degraded by loss of atrous convolution. Compared with those in UCM, AID, and NWPU45, the images in PatternNet cover fewer background regions. For the examples of basketball courts shown in Fig. 13(c)-(d), the basketball courts and storage tanks

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 08:14:10 UTC from IEEE Xplore. Restrictions apply.

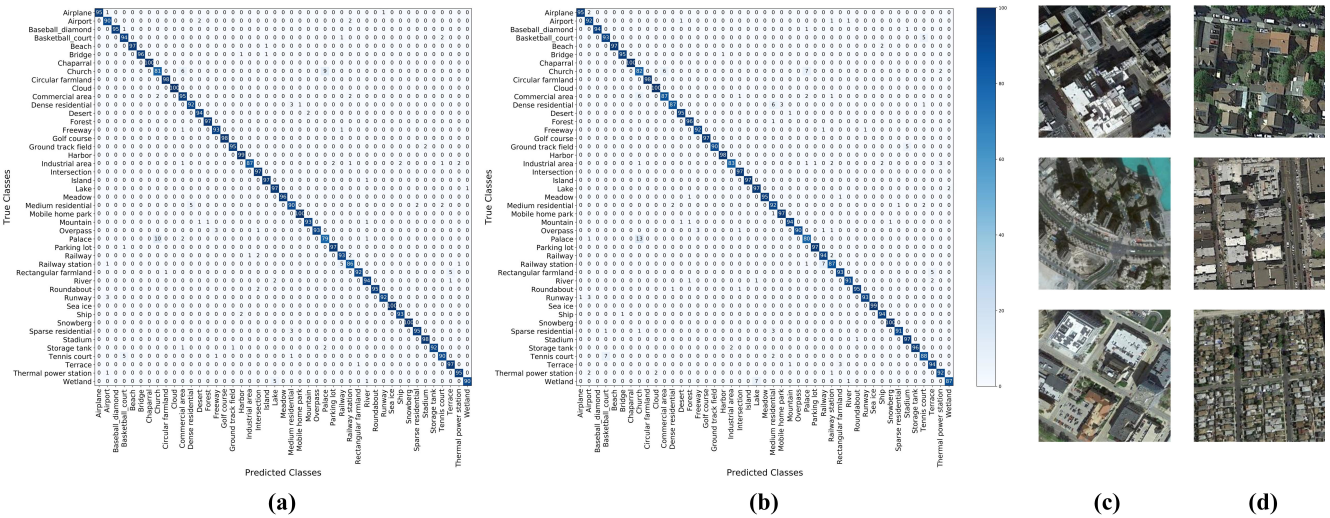

Fig. 12. Confusion matrix and image examples of the NWPU45 data set. (a) Confusion matrix of CIPAL. (b) Confusion matrix of ablation C. (c) Examples in commercial area class. (d) Examples in dense residential class.

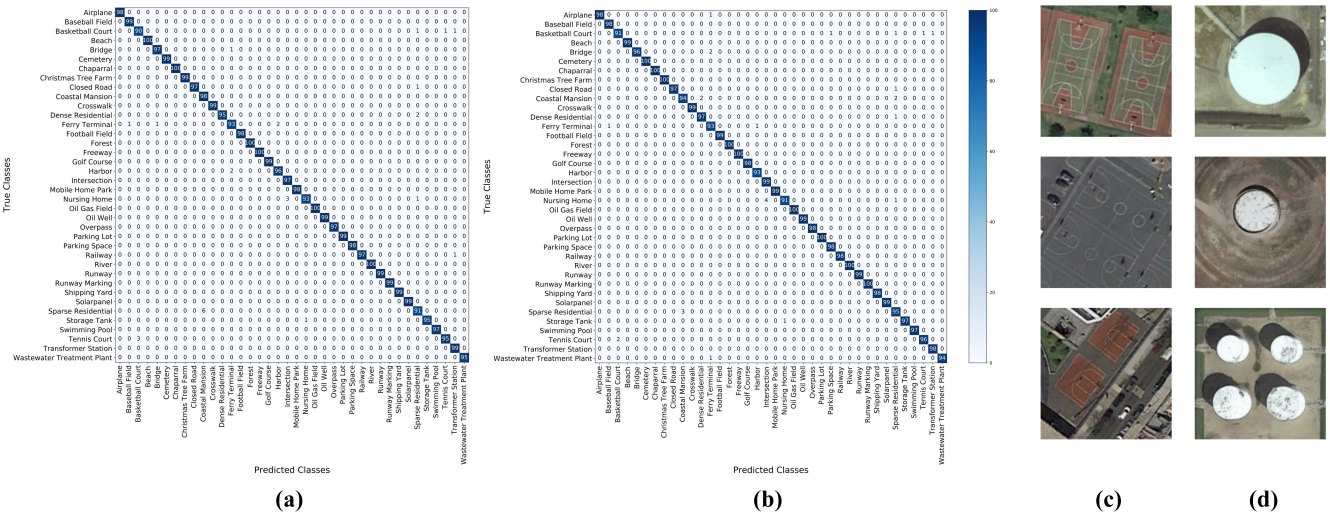

Fig. 13. Confusion matrix and image examples of the PatternNet data set. (a) Confusion matrix of CIPAL. (b) Confusion matrix of ablation C. (c) Examples in basketball court class. (d) Examples in storage tank class.

take up most of the pixels in the images. Thus, the scene classification task on PatternNet is less dependent on the larger receptive field than that on the other three data sets, resulting in a smaller accuracy decrease than that of other data sets in ablation C.

V. Conclusion

In this article, we proposed an architecture learning framework with channel compression that can learn a task-specific CNN architecture for remote-sensing scene classification. To utilize the contextual information in RSIs as much as possible, we learned architectures on larger resolution of RSIs and introduce a larger architecture space. In addition, a gradient descent-based learning strategy and channel compression for convolution operators help to reduce the memory and calculation consumption during the architecture learning procedure. The experimental results show that the task-specific CNN architecture outperforms most popular classic human-designed architectures on four remote-sensing scene classification benchmarks.

The architectures determine the function of CNNs, and our proposed framework helps in understanding which representations are important for remote-sensing scene classification tasks and may lead to a better foundation for remote-sensing scene classification. In future works, we will explore more operators potentially applicable to remote-sensing scene classification and other ways to make better use of contextual information in RSIs in architecture learning.

References

[1] A. Plaza, J. Plaza, A. Paz, and S. Sánchez, "Parallel hyperspectral image and signal processing," *IEEE Signal Process. Mag.*, vol. 28, no. 3, pp. 119–126, May 2011.

[2] H. M. J. Cantalloube and C. E. Nahum, "Airborne SAR-efficient signal processing for very high resolution," *Proc. IEEE*, vol. 101, no. 3, pp. 784–797, Mar. 2013.

[3] G. Cheng, J. Han, and X. Lu, "Remote sensing image scene classification: Benchmark and state of the art," *Proc. IEEE*, vol. 105, no. 10, pp. 1865–1883, Oct. 2017.

[4] G. Cheng, J. Han, P. Zhou, and L. Guo, "Multi-class geospatial object detection and geographic image classification based on collection of part detectors," *ISPRS J. Photogramm. Remote Sens.*, vol. 98, pp. 119–132, Dec. 2014.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 08:14:10 UTC from IEEE Xplore. Restrictions apply.

[5] Y. Wang *et al.*, "A three-layered graph-based learning approach for remote sensing image retrieval," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 10, pp. 6020–6034, Oct. 2016.

[6] Q. Zhu, Y. Zhong, B. Zhao, G.-S. Xia, and L. Zhang, "Bag-of-visual-words scene classifier with local and global features for high spatial resolution remote sensing imagery," *IEEE Geosci. Remote Sens. Lett.*, vol. 13, no. 6, pp. 747–751, Jun. 2016.

[7] R. C. Estoque, Y. Murayama, and C. M. Akiyama, "Pixel-based and object-based classifications using high-and medium-spatial-resolution imageries in the urban and suburban landscapes," *Geocarto Int.*, vol. 30, no. 10, pp. 1113–1129, 2015.

[8] T. Blaschke, "Object based image analysis for remote sensing," *ISPRS J. Photogramm. Remote Sens.*, vol. 65, no. 1, pp. 2–16, Jan. 2010.

[9] T. Blaschke *et al.*, "Geographic object-based image analysis–towards a new paradigm," *ISPRS J. Photogramm. Remote Sens.*, vol. 87, pp. 180–191, 2014.

[10] Y. Yang and S. Newsam, "Geographic image retrieval using local invariant features," *IEEE Trans. Geosci. Remote Sens.*, vol. 51, no. 2, pp. 818–832, Feb. 2013.

[11] N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," in *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2005, pp. 886–893.

[12] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, p. 436, 2015.

[13] G. J. Scott, M. R. England, W. A. Starms, R. A. Marcum, and C. H. Davis, "Training deep convolutional neural networks for land–cover classification of high-resolution imagery," *IEEE Geosci. Remote Sens. Lett.*, vol. 14, no. 4, pp. 549–553, Apr. 2017.

[14] E. Li, J. Xia, P. Du, C. Lin, and A. Samat, "Integrating multilayer features of convolutional neural networks for remote sensing scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 10, pp. 5653–5665, Oct. 2017.

[15] M. E. Paoletti, J. M. Haut, J. Plaza, and A. Plaza, "A new deep convolutional neural network for fast hyperspectral image classification," *ISPRS J. Photogramm. Remote Sens.*, vol. 145, pp. 120–147, Nov. 2018.

[16] K. Nogueira, O. A. B. Penatti, and J. A. dos Santos, "Towards better exploiting convolutional neural networks for remote sensing scene classification," *Pattern Recognit.*, vol. 61, pp. 539–556, Jan. 2017.

[17] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2012, pp. 1097–1105.

[18] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2014, *arXiv:1409.1556*. [Online]. Available: http://arxiv.org/abs/1409.1556

[19] Y. Yang and S. Newsam, "Bag-of-visual-words and spatial extensions for land-use classification," in *Proc. 18th SIGSPATIAL Int. Conf. Adv. Geographic Inf. Syst. (GIS)*, 2010, pp. 270–279.

[20] O. A. B. Penatti, K. Nogueira, and J. A. dos Santos, "Do deep features generalize from everyday objects to remote sensing and aerial scenes domains?" in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Jun. 2015, pp. 44–51.

[21] B. Zoph and Q. V. Le, "Neural architecture search with reinforcement learning," 2016, *arXiv:1611.01578*. [Online]. Available: http://arxiv.org/abs/1611.01578

[22] B. Zoph, V. Vasudevan, J. Shlens, and Q. V. Le, "Learning transferable architectures for scalable image recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 8697–8710.

[23] E. Real *et al.*, "Large-scale evolution of image classifiers," in *Proc. 34th Int. Conf. Mach. Learn.*, vol. 70, 2017, pp. 2902–2911.

[24] G. Bender, P.-J. Kindermans, B. Zoph, V. Vasudevan, and Q. Le, "Understanding and simplifying one-shot architecture search," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 550–559.

[25] M. Zhang *et al.*, "One-shot neural architecture search: Maximising diversity to overcome catastrophic forgetting," *IEEE Trans. Pattern Anal. Mach. Intell.*, early access, Nov. 3, 2020, doi: 10.1109/TPAMI.2020.3035351.

[26] H. Pham, M. Guan, B. Zoph, Q. Le, and J. Dean, "Efficient neural architecture search via parameters sharing," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 4095–4104.

[27] H. Liu, K. Simonyan, and Y. Yang, "DARTS: Differentiable architecture search," 2018, *arXiv:1806.09055*. [Online]. Available: http://arxiv.org/abs/1806.09055

[28] X. Chen, L. Xie, J. Wu, and Q. Tian, "Progressive differentiable architecture search: Bridging the depth gap between search and evaluation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 1294–1303.

[29] L.-C. Chen, G. Papandreou, I. Kokkinos, K. Murphy, and A. L. Yuille, "DeepLab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected CRFs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 40, no. 4, pp. 834–848, Apr. 2018.

[30] R. Kemker, C. Salvaggio, and C. Kanan, "Algorithms for semantic segmentation of multispectral remote sensing imagery using deep learning," *ISPRS J. Photogramm. Remote Sens.*, vol. 145, pp. 60–77, Nov. 2018.

[31] G. Chen, X. Zhang, Q. Wang, F. Dai, Y. Gong, and K. Zhu, "Symmetrical dense-shortcut deep fully convolutional networks for semantic segmentation of very-high-resolution remote sensing images," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 11, no. 5, pp. 1633–1644, May 2018.

[32] X.-Y. Tong *et al.*, "Land-cover classification with high-resolution remote sensing images using transferable deep models," *Remote Sens. Environ.*, vol. 237, Feb. 2020, Art. no. 111322.

[33] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.

[34] C. Szegedy *et al.*, "Going deeper with convolutions," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2015, pp. 1–9.

[35] S. Bhagavathy and B. S. Manjunath, "Modeling and detection of geospatial objects using texture motifs," *IEEE Trans. Geosci. Remote Sens.*, vol. 44, no. 12, pp. 3706–3715, Dec. 2006.

[36] V. Risojevic and Z. Babic, "Fusion of global and local descriptors for remote sensing image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 10, no. 4, pp. 836–840, Jul. 2013.

[37] J. Zou, W. Li, C. Chen, and Q. Du, "Scene classification using local and global features with collaborative representation fusion," *Inf. Sci.*, vol. 348, pp. 209–226, Jun. 2016.

[38] M. Castelluccio, G. Poggi, C. Sansone, and L. Verdoliva, "Land use classification in remote sensing images by convolutional neural networks," 2015, *arXiv:1508.00092*. [Online]. Available: http://arxiv.org/abs/1508.00092

[39] G.-S. Xia *et al.*, "AID: A benchmark data set for performance evaluation of aerial scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 7, pp. 3965–3981, Jul. 2017.

[40] S. Chaib, H. Liu, Y. Gu, and H. Yao, "Deep feature fusion for VHR remote sensing scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 8, pp. 4775–4784, Aug. 2017.

[41] X. Liu, Y. Zhou, J. Zhao, R. Yao, B. Liu, and Y. Zheng, "Siamese convolutional neural networks for remote sensing scene classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 16, no. 8, pp. 1200–1204, Aug. 2019.

[42] Q. Bi, K. Qin, Z. Li, H. Zhang, K. Xu, and G.-S. Xia, "A multiple-instance densely-connected ConvNet for aerial scene classification," *IEEE Trans. Image Process.*, vol. 29, pp. 4911–4926, 2020.

[43] C. Ma, X. Mu, R. Lin, and S. Wang, "Multilayer feature fusion with weight adjustment based on a convolutional neural network for remote sensing scene classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 18, no. 2, pp. 241–245, Feb. 2021.

[44] J. Kang, R. Fernandez-Beltran, D. Hong, J. Chanussot, and A. Plaza, "Graph relation network: Modeling relations between scenes for multilabel remote-sensing image classification and retrieval," *IEEE Trans. Geosci. Remote Sens.*, early access, Aug. 21, 2020, doi: 10.1109/TGRS.2020.3016020.

[45] W. Zhang, P. Tang, and L. Zhao, "Remote sensing image scene classification using CNN-CapsNet," *Remote Sens.*, vol. 11, no. 5, p. 494, Feb. 2019.

[46] Q. Wang, S. Liu, J. Chanussot, and X. Li, "Scene classification with recurrent attention of VHR remote sensing images," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 2, pp. 1155–1167, Feb. 2019.

[47] W. Tong, W. Chen, W. Han, X. Li, and L. Wang, "Channel-attention-based DenseNet network for remote sensing image scene classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 13, pp. 4121–4132, 2020.

[48] R. Cao, L. Fang, T. Lu, and N. He, "Self-attention-based deep feature fusion for remote sensing scene classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 18, no. 1, pp. 43–47, Jan. 2021.

[49] D. Hong, L. Gao, J. Yao, B. Zhang, A. Plaza, and J. Chanussot, "Graph convolutional networks for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, early access, Aug. 18, 2020, doi: 10.1109/TGRS.2020.3015157.

[50] D. Hong *et al.*, "More diverse means better: Multimodal deep learning meets remote-sensing imagery classification," *IEEE Trans. Geosci. Remote Sens.*, early access, Aug. 24, 2020, doi: 10.1109/TGRS.2020.3016820.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 08:14:10 UTC from IEEE Xplore. Restrictions apply.

[51] B. Zhang, Y. Zhang, and S. Wang, "A lightweight and discriminative model for remote sensing scene classification with multidilation pooling module," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 12, no. 8, pp. 2636–2653, Aug. 2019.
[52] L. Li *et al.*, "A multiscale self-adaptive attention network for remote sensing scene classification," *Remote Sens.*, vol. 12, no. 14, p. 2209, Jul. 2020.
[53] Z. Zhao, J. Li, Z. Luo, J. Li, and C. Chen, "Remote sensing image scene classification based on an enhanced attention module," *IEEE Geosci. Remote Sens. Lett.*, early access, Aug. 4, 2020, doi: 10.1109/LGRS.2020.3011405.
[54] E. Real, A. Aggarwal, Y. Huang, and Q. V. Le, "Regularized evolution for image classifier architecture search," in *Proc. AAAI Conf. Artif. Intell.*, vol. 33, 2019, pp. 4780–4789.
[55] S. Xie, A. Kirillov, R. Girshick, and K. He, "Exploring randomly wired neural networks for image recognition," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 1284–1293.
[56] L. Li and A. Talwalkar, "Random search and reproducibility for neural architecture search," 2019, *arXiv:1902.07638*. [Online]. Available: http://arxiv.org/abs/1902.07638
[57] Y. Xu *et al.*, "PC-DARTS: Partial channel connections for memory-efficient architecture search," in *Proc. Int. Conf. Learn. Represent.*, 2019, pp. 1–13.
[58] Y. Chen, K. Zhu, L. Zhu, X. He, P. Ghamisi, and J. A. Benediktsson, "Automatic design of convolutional neural network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 9, pp. 7048–7066, Sep. 2019.
[59] J. Wang, Y. Zhong, Z. Zheng, A. Ma, and L. Zhang, "RSNet: The search for remote sensing deep neural networks in recognition tasks," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 3, pp. 2520–2534, Mar. 2021.
[60] W. Zhou, S. Newsam, C. Li, and Z. Shao, "PatternNet: A benchmark dataset for performance evaluation of remote sensing image retrieval," *ISPRS J. Photogramm. Remote Sens.*, vol. 145, pp. 197–209, Nov. 2018.
[61] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 2818–2826.
[62] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 4700–4708.
[63] F. Yu and V. Koltun, "Multi-scale context aggregation by dilated convolutions," 2015, *arXiv:1511.07122*. [Online]. Available: http://arxiv.org/abs/1511.07122

Jie Chen (Member, IEEE) received the M.S. degree in cartography and geographic information engineering from the East China University of Technology, Nanchang, China, and the Ph.D. degree in photogrammetry and remote sensing from Central South University, Changsha, China, in 2006 and 2010, respectively.

He is an Associate Professor with the Department of Geo-informatics, Central South University. His research interests include intelligent analysis and understanding of RSIs.

Dr. Chen is a Reviewer for several journals in the field of remote sensing.

Haozhe Huang received the B.S. degree in remote-sensing science and technology from Wuhan University, Wuhan, China. He is pursuing the Ph.D. degree with the School of Geosciences and Info-Physics, Central South University, Changsha, China.

His research interests include understanding of remote-sensing images and deep learning.

Jian Peng received the B.S. degree in 2015. He is pursuing the Ph.D. degree with the School of Geosciences and Info-Physics, Central South University, Changsha, China.

His research interests include memory models, deep learning, and artificial intelligence.

Jiawei Zhu received the B.S. degree in geographic information system and the M.S. degree in cartography and geographic information science from Central South University, Changsha, China, in 2013 and 2016, respectively, where she is pursuing the Ph.D. degree in surveying and mapping.

Her research interests include network representation learning, topological data analysis, and spatiotemporal data mining.

Li Chen received the B.Sc. and M.Sc. degrees from the School of Software, Central South University, Changsha, China, in 2015 and 2018, respectively, where he is pursuing the Ph.D. degree with the School of Geosciences and Info-Physics.

His research interests include transfer learning, deep learning, and visualization.

Chao Tao received the B.S. degree from the School of Mathematics and Statistics, Huazhong University of Science and Technology, Wuhan, China, in 2007, and the Ph.D. degree from the Institution of Patter Recognition and Artificial Intelligence, Huazhong University of Science and Technology, Wuhan, China, in 2012.

He is an Associate Processor with the School of Geosciences and Info-Physics, Central South University, Changsha, China. He has authored more than 30 peer-reviewed articles in international journals from multiple domains such as remote sensing and computer vision. His research interests include computer vision, machine learning, deep learning, and their applications in remote sensing.

Dr. Tao has been frequently serving as a Reviewer for more than four international journals, including the IEEE TRANSACTIONS ON GEOSCIENCE AND REMOTE SENSING (TGRS), the IEEE GEOSCIENCE AND REMOTE SENSING LETTERS (GRSL), the IEEE JOURNAL OF SELECTED TOPICS IN APPLIED EARTH OBSERVATIONS AND REMOTE SENSING (JSTAR), PERS and ISPRS–JPRS. He is also a Communication Evaluation Expert for the National Natural Science Foundation of China.

Haifeng Li (Member, IEEE) received the master's degree in transportation engineering from the South China University of Technology, Guangzhou, China, in 2005, and the Ph.D. degree in photogrammetry and remote sensing from Wuhan University, Wuhan, China, in 2009.

He was a Research Associate with the Department of Land Surveying and Geo-Informatics, The Hong Kong Polytechnic University, Hong Kong, in 2011, and a Visiting Scholar with the University of Illinois at Urbana-Champaign, Urbana, IL, USA, from 2013 to 2014. He is a Professor with the School of Geosciences and Info-Physics, Central South University, Changsha, China. He has authored over 30 journal articles. His research interests include geo/remote-sensing big data, machine/deep learning, and artificial/brain-inspired intelligence.

Dr. Li is a Reviewer for many journals.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 08:14:10 UTC from IEEE Xplore. Restrictions apply.